

Building an Embedded System Based on the Initial RAM Disk

Here's a rough tutorial on writing an embedded Linux based system, using an initial RAM disk.



The initial RAM disk (*initrd*) is a tool for loading a temporary root file system into the memory during the Linux boot process. The files of the system perform some initialisation jobs. A boot loader loads the Linux kernel and the *initrd* into the memory, starts the kernel and informs it about the location of the *initrd*. The kernel mounts the *initrd* as its initial root file system at the end of its boot sequence. Then the kernel executes the */linuxrc* file as its first process. When the */linuxrc* file exits, the kernel assumes that it has mounted the final root file system.

Today, there are more modern methods to achieve this goal—*initramfs* and *cramfs*, though I've never tried them.

The idea is to put an embedded application as the */linuxrc* file into the *initrd*. The system will never go beyond its *initrd* stage. No Linux daemons will haunt the embedded application.

This works fine for me but there is a disadvantage.

The embedded application is resident twice in the embedded computer memory: first as a part of the *initrd* file system and second, as a running process. My embedded application is a small one but it needs a substantial amount of numerical data. I put the data into files outside the *initrd* on the embedded computer's disk. When the application starts, it reads the data from the files into the memory. If your embedded computer has enough memory, you can use the presented method directly. If not, you may want to investigate how to run an executable located outside the *initrd* file system. I've never tried this but I believe it is possible.

So let's get started with the tutorial.

Configuring the kernel for your embedded computer

Keep your kernel small, simple and compact. Include only